UNIVERSITY OF PASSAU

Faculty of Computer Science and Mathematics

Reverse Engineering



Prof. Dr.-Ing. Christian Hammer

WS 2026/27

Binary Executable Formats

- Natural candidate for static analysis of target binary
- Identify machine code to be run and other artifacts
- Metadata: libraries and functions referenced from binary at runtime potentially exported functionalities

- Platform dependent
- Unix: ELF Executable and Linkable Format
  origin: Unix System V Release 4
  Linux, Unix variants, Game consoles, Mobile phones and others
- Win: PE Portable Executable Format
  also for Unified Extensible Firmware Interface (UEFI) (successor BIOS)
- Mac: Mach object file format
- Legacy: com, dos MZ, a.out, COFF

- Source code compiled to object files (e.g. .o or .obj)
- Stored in some executable format (specified or implicit for compiler)
- (Static) linking integrates object files into executable
- Goal: Relocation of code and data in virtual memory
- Dynamic linker (OS) needs to load external libraries into shared memory space

Loading Executable

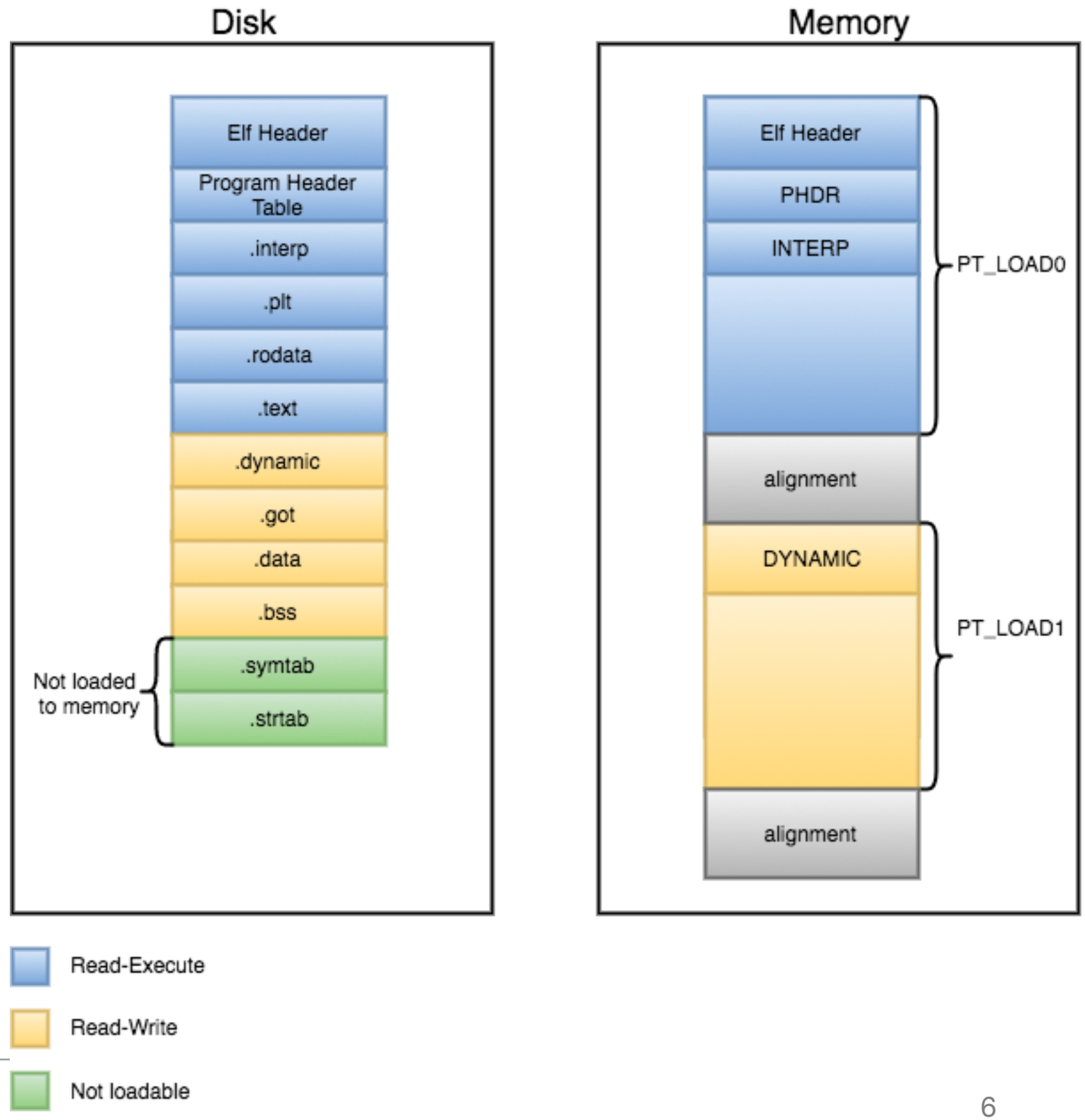
- Kernel and OS functionality when starting an executable
- Creation of new OS process
- Loading of program image into memory of this process
- (Recurs.) resolving runtime dependencies (dynamic libraries)
- Relocation: patch absolute/relative virtual addresses and resolve addresses of imported functions
- Invoke shared library initialization functions
- Jump to entry point

ELF Format

- Cross-Platform
- Header (0x7f ELF) defines 32 and 64 bit, endianness, OS ABI, ISA, pointers, size and numbers of Program header table/entries
- Program Header Table: segments of the executable
  loadable segment, dynamic linking info, interpreter info, thread-local storage
- Section Header Table: sections important for linking and execution
  provides a mapping to virtual memory
  program, symbols, strings, dynamic linking info, constructors, destructors,
  (e.g. .text: read-execute, .data: read-write, no-load)
- Tools: readelf -h <exec>, elfutils, objdump, file

Binary Loading with Segments and Sections

- Segments and virtual addresses must be aligned to page size!



Common Sections

- .text: code.
- .data: initialised data.
- .rodata: initialised read-only data.
- .bss: uninitialized data.
- .plt: PLT (Procedure Linkage Table) (IAT equivalent).
- .got: GOT entries dedicated to dynamically linked global variables.
- .got.plt: GOT entries dedicated to dynamically linked functions.
- .symtab: global symbol table.
- .dynamic: Holds all needed information for dynamic linking.
- .dynsym: symbol tables dedicated to dynamically linked symbols.
- .strtab: string table of .symtab section.
- .dynstr: string table of .dynsym section.
- .interp: RTLD embedded string.
- .rel.dyn: global variable relocation table.
- .rel.plt: function relocation table.

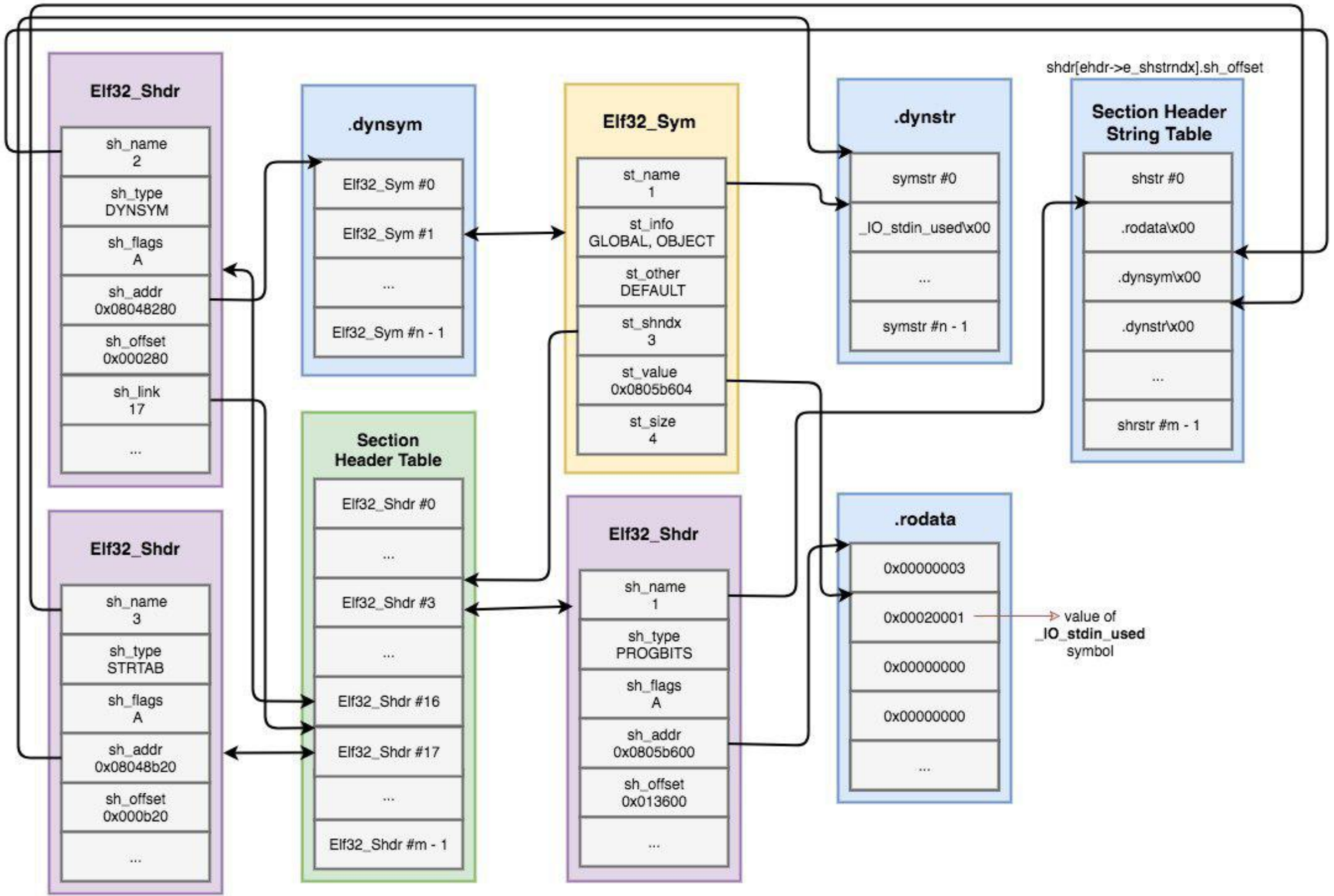
- PT_NULL: unassigned segment (usually first entry of Program Header Table).
- PT_LOAD: Loadable segment (into memory), all others are mapped into one of these!
- PT_INTERP: Segment holding .interp section.
- PT_TLS: Thread Local Storage segment (Common in statically linked binaries).
- PT_DYNAMIC: Holding .dynamic section.

- Different segments usually have different access attributes
- readelf -l <exec>

- Compiler generates more than just code
- Symbolic info from source can improve interpretation of machine code
- Linkers refer to symbol table (match/reference/modify a symbol)
- Debuggers need symbols for better matching of names (vars/functions)
- Anti-analysis tools remove as many symbols as possible
- st_name: index into symbol name string table
- st_info: symbol binds (linker visibility: local, global, (global) weak)
  types: object (variable), func (function), section (ELF section)
- st_shndx: associated section type
  undef: external, progbits: in current object, symtab: symbol table, …

- Max 2 symbol tables per ELF object (.symtab, .dynsym)
- .symtab: binary's global symbol table
- section .strtab for (null-terminated) Strings referencing objects from a different section, one for each symbol of .symtab
- .dynsym required for dynamic linking of external objects corresponding string table called .dynstr
- .symtab can be removed, .dynsym cannot unless no dynamic linking

Symbols Example



- Dynamic linker (“interpreter”) is loaded first
- Sets up environment (recursively) using entries from .dynamic section
  - DT_NEEDED – name of required dependency to load.
  - DT_SYMTAB – address of dynamic symbol table.
  - DT_FLAGS – e.g. BIND_NOW flag
- Perform relocations
- Linker loads main executable and (eager) dependencies, invokes library initialization(s)
- Finally transfers control to executable’s entry point

- Resolve function references only when needed
- Global Offset Table (GOT): addresses of global variables and external functions
- Procedure Linkage Table (PLT): short entries of instructions (“trampolines” or “thunks”): jump to corresponding GOT entry
- Eager linking: GOT entries relocated at load-time
- Lazy linking: GOT entry determined (through loading of library) and relocated at first use, initially GOT entry points to dynamic linker

Summary

- Executable files store significant metadata alongside binary
- Metadata for dynamic linking cannot be stripped
- The more metadata can be found the easier semantics can be inferred

- Ignacio Sanmillan, Paul Litvak: Executable and Linkable Format 101 https://www.intezer.com/blog/research/executable-linkable-format-101-part1-sections-segments/ – https://www.intezer.com/blog/malware-analysis/executable-linkable-format-101-part-4-dynamic-linking/
- Ryan O'Neill, Learning Linux binary analysis, packt publishing, 2016
- J. Kinder: Reverse Engineering, Lecture at UBW, 2020
- Microsoft Documentation for Visual Studio 2022